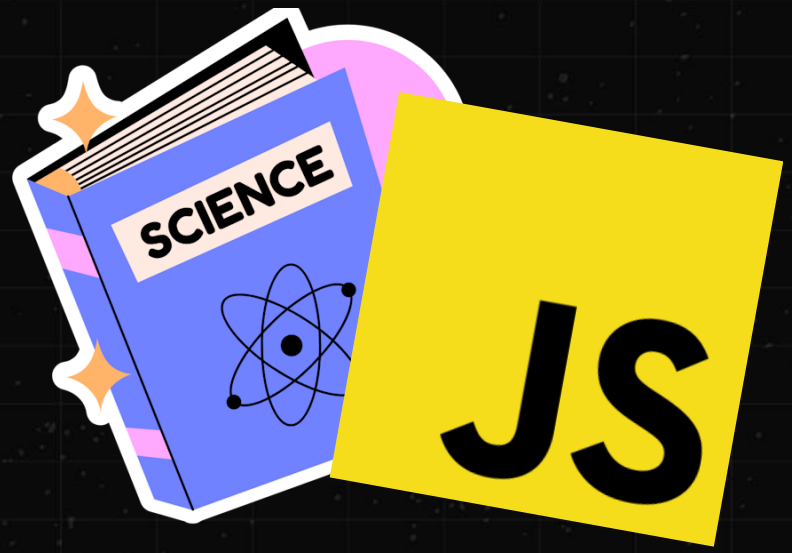
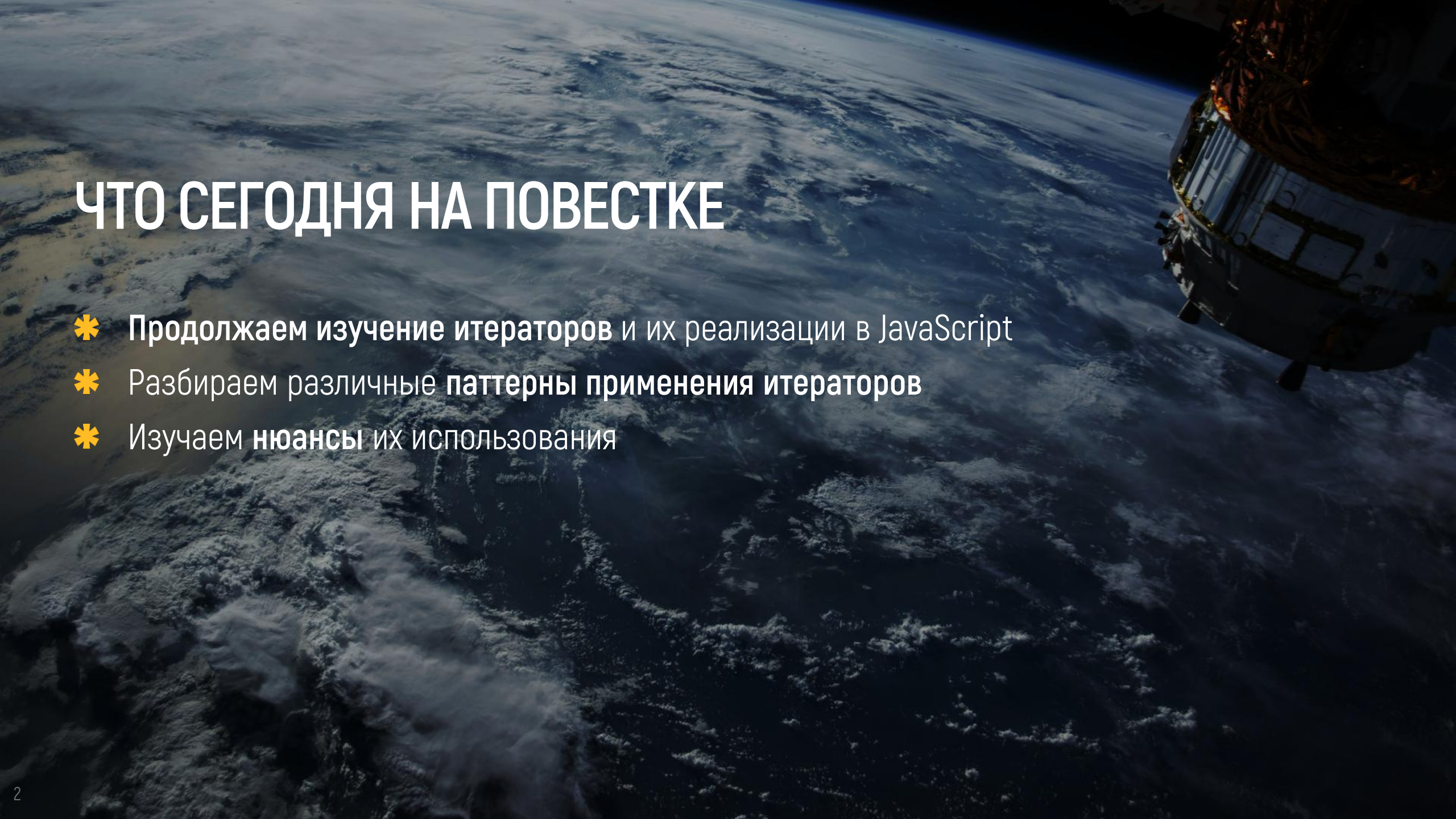
KOBEZZA.LAB

ЛЕКЦИЯ #24

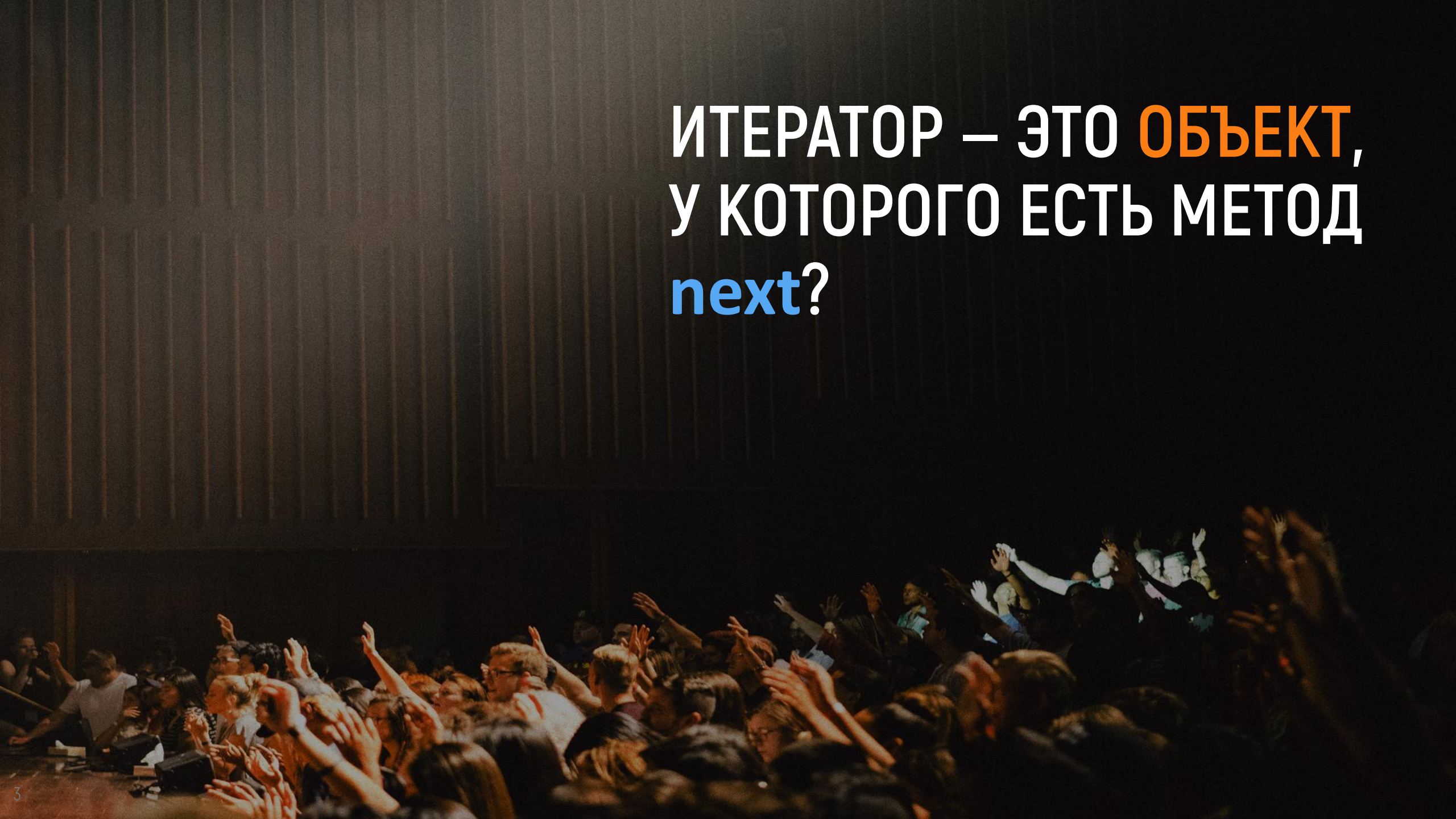
CS BO FRONTEND





ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- **Продолжаем изучение итераторов** и их реализации в JavaScript
- Разбираем различные **паттерны применения итераторов**
- Изучаем **нюансы** их использования


ИТЕРАТОР — ЭТО ОБЪЕКТ,
У КОТОРОГО ЕСТЬ МЕТОД
next?

НЕ СОВСЕМ ТАК

- **Названия методов, а также их количество может отличаться** в зависимости от конкретной реализации
- Как и то, **что возвращают эти методы**
- Но главное – **итератор должен уметь как-то «двигаться» по данным,** для которых он создан
- И уметь **сигнализировать, что данные закончились**

ЭТО УНИВЕРСАЛЬНЫЙ ПАТТЕРН

- На первый взгляд, итераторы неотрывно связаны с коллекциями данных
- В действительности, **итератор можно создать для чего угодно**
- Например, для генерации бесконечной последовательности случайных чисел
- Или символов в заданном диапазоне

```
// 1. Бесконечная последовательность случайных чисел
function randomNumbers() {
   return Iterator.from({ next: () => ({ value: Math.random(), done: false }) });
}

const randoms = randomNumbers();

console.log(randoms.next().value); // Случайное число
console.log(randoms.next().value); // Следующее случайное

// 2. Символы в заданном диапазоне
function charRange(start, end) {
   let code  = start.codePointAt(0) - 1;
   let endCode  = end.codePointAt(0);

   return Iterator.from({
    next() {
     if (code >= endCode) {
        return { value: undefined, done: true };
     }

     code++;
     return { value: String.fromCodePoint(code), done: false };
    }
   });
}

const letters = charRange("a", "e");
console.log([...letters]); // [ 'a', 'b', 'c', 'd', 'e' ]
```

ЧТО ЕЩЕ ЗА *Iterator*.from?



ИТЕРИРУЕМЫЕ (ITERABLE) ОБЪЕКТЫ

- Итераторы интегрируются с другими синтаксическими конструкциями языка **не напрямую, а через абстракцию итерируемых объектов**
- Это такие объекты, для которых **есть стандартный способ создать итератор**
- В JavaScript такой способ выражается **в наличии метода Symbol.iterator**
- **Итераторы тоже должны быть итерируемыми,** чтобы мы могли использовать их вместе с циклом for of или оператором spread (...)
- Поэтому итератор тоже должен иметь метод Symbol.iterator, который просто возвращает ссылку на себя (this)

```
// Итератор — тоже итерируемый объект
function createRange(start, end) {
  let current = start;

  return {
    // ✅ Итератор должен быть iterable
    [Symbol.iterator]() {
      return this; // Возвращает себя
    },

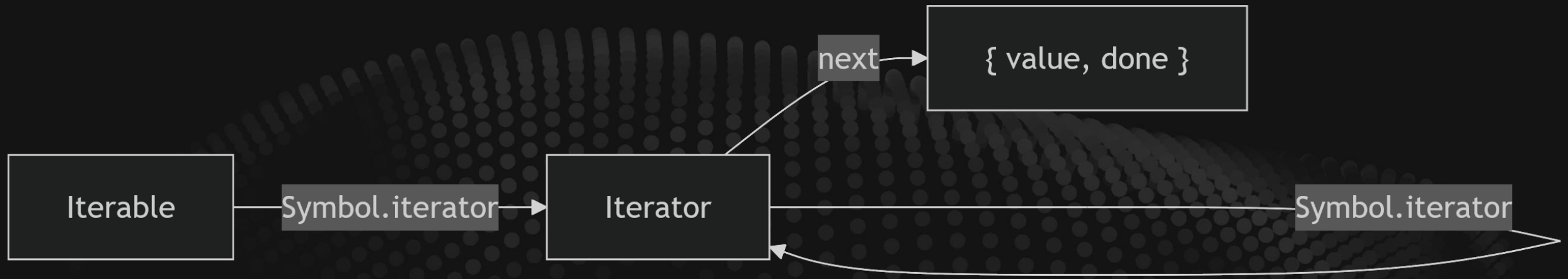
    next() {
      if (current > end) {
        return { value: undefined, done: true };
      }

      return { value: current++, done: false };
    },
  };
}

// ✅ Работает с for...of
for (const num of createRange(1, 3)) {
  console.log(num); // 1, 2, 3
}

// ✅ Работает с spread
console.log([...createRange(1, 3)]); // [1, 2, 3]
```

ВИЗУАЛИЗИРУЕМ



ЛЕГКО ЗАБЫТЬ И ОШИБИТЬСЯ

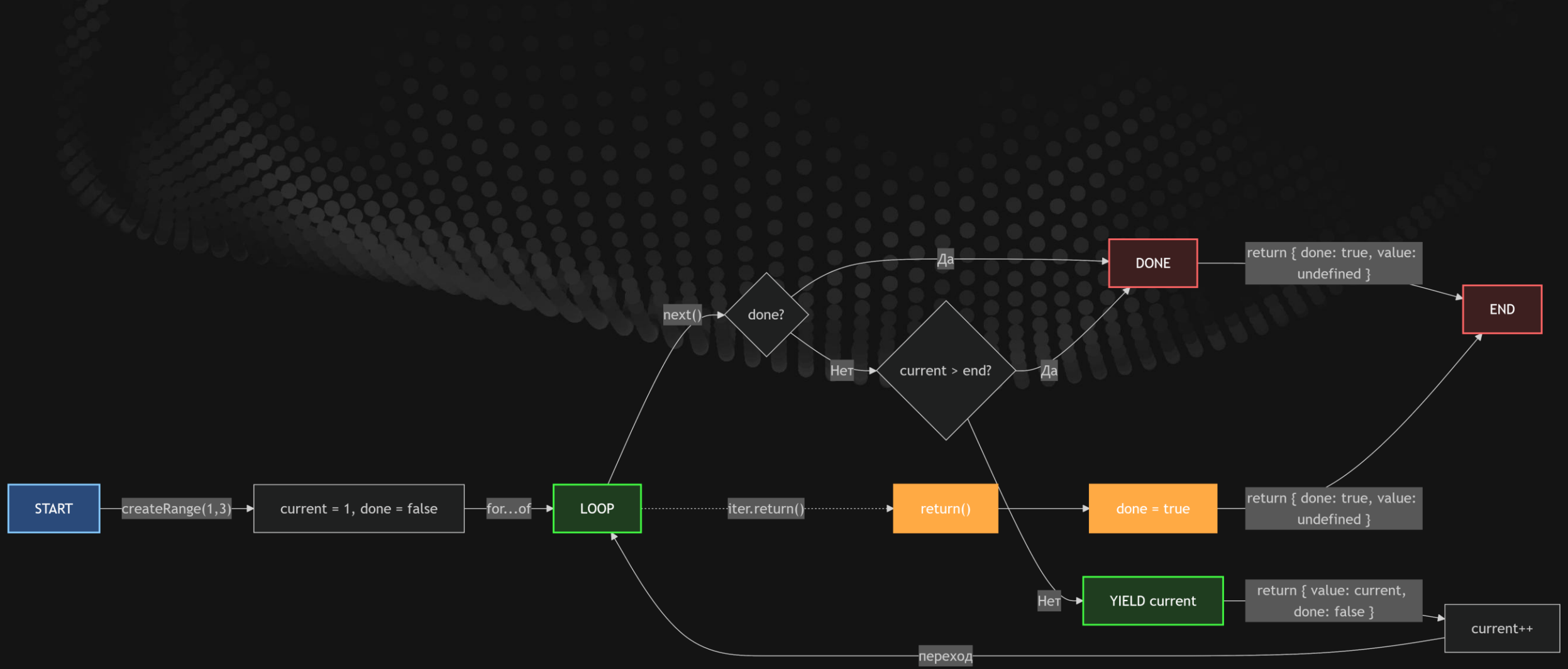
- Поэтому можно использовать фабрику *Iterator*.from, которая создаст валидный **итератор со всеми необходимыми методами на основе обычного объекта**
- В качестве аргумента достаточно **передать объект с одним методом next**



А КАКИЕ ЕЩЕ ТАМ ЕСТЬ МЕТОДЫ?

НЕ ТОЛЬКО next

- В JS итератор может также определять метод return
- Вызов return **имеет семантику «остановить» итератор** и должен возвращать объект { value, done: true }


START
createRange(1,3)
current = 1, done = false
for...of
LOOP
next()
done?
Да
DONE
return { done: true, value: undefined }
END
Нет
current > end?
Да
iter.return()
return()
done = true
return { done: true, value: undefined }
Нет
YIELD current
return { value: current, done: false }
current++
переход

РЕФЛЕКСИРУЕМ

- Итератор – это **ещё один пример автомата**
- В нашем случае мы **добавили терминальное состояние** `done`, которое устанавливается в момент вызова `return`
- Ещё одна особенность этого метода в том, что он вызывается автоматически, **когда в теле цикла происходит досрочный выход**
- Например, при вызове `break`, `return` или `throw` (который прервал цикл)

```
function createRange(start, end) {
  let current = start;
  let done = false;

  return Iterator.from({
    return(value) {
      done = true;
      return { value, done };
    },

    next() {
      if (done || current > end) {
        return { value: undefined, done: true };
      }

      return { value: current++, done: false };
    },
  });
}

const iter = createRange(1, 3);

for (const num of iter) {
  console.log(num); // 1
  break;                  // Вызов iter.return()
}

console.log([...iter]); // []
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Метод return как бы говорит итератору: «Тебе пора прерваться»
- Когда в цикле происходит досрочный выход,
  то **если у итератора есть такой метод, он всегда будет вызываться**
- Явно определяя этот метод, **вы контролируете состояние вашего автомата**

А ЕСЛИ МЕТОДА НЕТ?



НИЧЕГО НЕ ПРОИЗОЙДЕТ

- Цикл всё равно прервётся, но ваш итератор не изменит своё состояние
- Более того, если вы передали в *Iterator*.from итератор без метода return, то он **создаст его автоматически, но он не будет ничего делать**

```
function createRange(start, end) {
  let current = start;
  let done = false;

  return Iterator.from({
    next() {
      if (done || current > end) {
        return { value: undefined, done: true };
      }

      return { value: current++, done: false };
    },
  });
}

const iter = createRange(1, 3);

console.log(iter.return()); // { value: undefined, done: true }

for (const num of iter) {
  console.log(num); // 1
  break;                  // Вызов iter.return();
}

console.log([...iter]); // [ 2, 3 ]
```

РЕФЛЕКСИРУЕМ

- **Нет строгого требования,** что вызов return должен приводить к тому, что все последующие вызовы next возвращают { done: true }
- Как и нет требования, что после { done: true } не может быть снова { done: false }
- Например, вы **можете создать итератор, который просто ходит по кругу** (бесконечный циклический итератор)

```
function createRange(start, end) {
  let current = start;

  return Iterator.from({
    next() {
      if (current > end) {
        current = start;
        return { value: undefined, done: true };
      }

      return { value: current++, done: false };
    },
  });
}

const iter = createRange(1, 3);

for (const num of iter) {
  console.log(num); // 1 2 3
}

for (const num of iter) {
  console.log(num); // 1 2 3
}

for (const num of iter) {
  console.log(num); // 1 2 3
}
```


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ЗАПОМНИТЕ

- Вы сами определяете, как автомату итератора **обрабатывать сигнал остановки и что после этого делать**
- Обычное поведение большинства итераторов – после вызова `return` **просто ничего не делать**
- А после того, **как данные закончились, всегда возвращать** `{ done: true }`
- Но есть много ситуаций, где удобнее сделать иначе


А ЕЩЕ ТАМ
ЕСТЬ МЕТОДЫ?

ДА, И ИХ МНОГО

- Некоторые из них **похожи на методы массивов**
- Например, filter или map
- В отличие от методов массива, которые немедленно выполняют заданную операцию и возвращают новое значение, **методы итератора возвращают новый итератор**
- Фактически работа начнётся только при вызове next

```
const iter = new Set([1, 2, 3, 4, 5]).values();

const arr = Array.from(
   iter
      .filter((el) => el > 3)
      .map((el) => el * 10)
);

// [ 40, 50 ]
console.log(arr);
```

РЕФЛЕКСИРУЕМ

- Встроенные итераторы в JS предоставляют набор методов-адаптеров, которые **на основе исходного итератора создают новый**
- Поскольку итераторы универсальны, нам **не важен конкретный источник данных**
- Это может быть массив, строка, Map, Set или **любой другой итерируемый объект**
- **Итераторы работают лениво:** фактически выполняется только необходимая работа

АДАПТЕРЫ ОПРЕДЕЛЯЮТ ТЕРМИНАЛЬНЫЙ return

- Это означает, что даже если исходный итератор не перешёл в терминальное состояние после вызова return, **адаптер будет считать итератор завершённым**
- При этом исходный итератор может быть по-прежнему «рабочим»

```
const a = [1, 2, 3, 4, 5];

const iter = a.values();

for (const el of iter) {
  console.log(el); // 1
  break;
}

for (const el of iter) {
  console.log(el); // 2
  break;
}

const iter2 = iter.filter((el) => el > 0);

for (const el of iter2) {
  console.log(el); // 3
  break;
}

// Итератор перешел в терминальное состояние из-за прошлого break
for (const el of iter2) {
  console.log(el);
  break;
}

// А исходный итератор по-прежнему работает
for (const el of iter) {
  console.log(el); // 4
  break;
}
```

ИТЕРАТОРЫ VS ФУНКЦИИ ВЫСШЕГО ПОРЯДКА

- Встроенные функции высшего порядка массива «энергичны»
- `[...].map().filter().map().filter().slice(0, 2)`
- Каждый метод **создает новую коллекцию в памяти** (с конкретным типом)
- Такие методы **есть не у всех коллекций**

ИТЕРАТОРЫ КАК СПОСОБ ОПТИМИЗАЦИИ

- При использовании «энергичных» методов **нередко выполняется лишняя работа** – в том числе обработка элементов, которые в итоге не понадобятся
- Простой **переход на итераторы способен дать заметное ускорение** без существенных изменений в коде
- Однако тут есть нюансы


ИТЕРАТОРЫ
РАБОТАЮТ
БЫСТРЕЕ?

ЕСТЬ НЮАНСЫ

- Алгоритмически итераторы зачастую эффективнее «энергичных» аналогов **при неполных операциях благодаря ленивой обработке данных**
- Они также **выигрывают в цепочках преобразований:** операции «схлопываются», и промежуточные коллекции не создаются
- Однако при полном проходе по всем элементам итераторы в JavaScript могут **уступать в производительности из-за особенностей реализации языка**


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- **Из-за особенностей реализации JS может наблюдаться заметная разница** в производительности между нативными методами итераторов и пользовательскими
- В ряде случаев самописные итераторы оказываются **существенно менее эффективными**
- На это влияют дополнительные накладные расходы на вызовы функций или отсутствие низкоуровневых оптимизаций движка
- **Про это нужно помнить** при работе с критическими по производительности участками кода

А ЕЩЕ ТАМ ЕСТЬ АДАПТЕРЫ?



ДА

- Кроме filter и map есть и **другие встроенные адаптеры**
- take(n) – берёт первые n элементов из итератора
- drop(n) – пропускает первые n элементов и возвращает остальные
- flatMap(fn) – применяет функцию fn к каждому элементу
- Если fn возвращает итерируемый объект (например, массив), flatMap «разворачивает» его в один поток значений

```
function random() {
    return Iterator.from({
        next() {
            return { value: Math.random(), done: false };
        }
    });
}

const iter = random()
  .filter((value) => value > 0.5)
  .map((value) => [value, value.toFixed(2)])
  .take(10);

// { value: [ 0.5611496791645234, '0.56' ], done: false }
console.log(iter.next());

const arr = Array.from(iter.flatMap((value) => value));

// [
//   0.6531659313989404, '0.65',
//   0.5686225221688315, '0.57',
//   0.9814406075901988, '0.98',
//   0.723446536222368,  '0.72',
//   0.7597448080711289, '0.76',
//   0.6416302703972017, '0.64',
//   0.7390271002647604, '0.74',
//   0.6122289857954435, '0.61',
//   0.5377850995521176, '0.54'
// ]
console.log(arr);
```

РЕФЛЕКСИРУЕМ

- **Главное, что важно понимать:** адаптеры итераторов лишь создают новый итератор на основе исходного
- Это позволяет **выстраивать гибкие композиции** операций и передавать полученные итераторы любому потребителю, ожидающему Iterable-объект

```
function random() {
  return Iterator.from({
    next() {
      return { value: Math.random(), done: false };
    }
  });
}

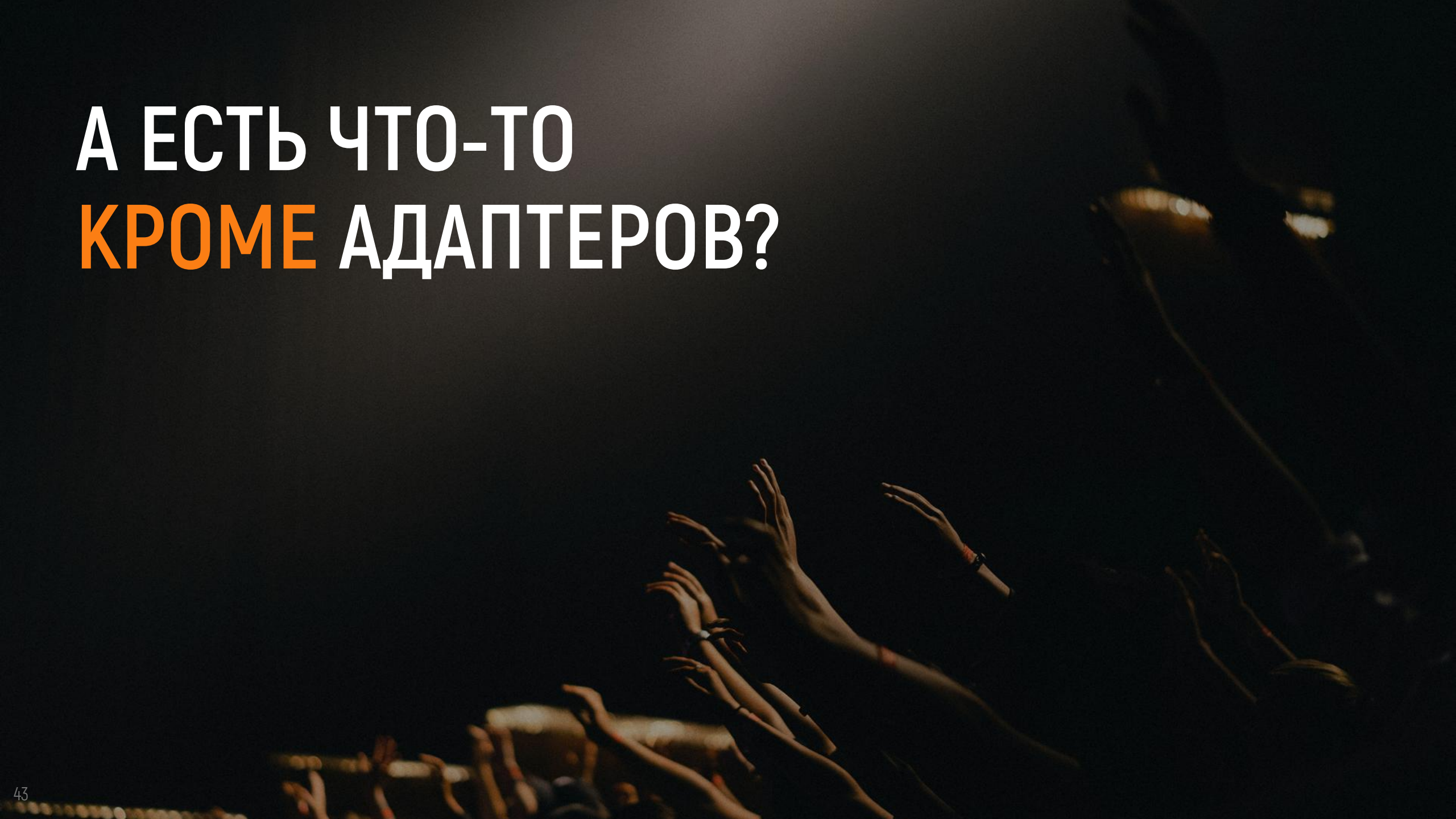
function zip(iterable1, iterable2) {
  // Iterator.from можно использовать и вместо iterable[Symbol.iterator]()
  const iter1 = Iterator.from(iterable1);
  const iter2 = Iterator.from(iterable2);

  return Iterator.from({
    next() {
      const result1 = iter1.next();
      const result2 = iter2.next();

      return {
        value: [result1.value, result2.value],
        done: result1.done || result2.done
      };
    }
  });
}

// [ 0.7420035533477523, 0.38943574765649125 ] [ 0.6572884870795378, 0.01976327038631731 ]
console.log(
  ...zip(
    random().filter((value) => value > 0.5),
    random().filter((value) => value < 0.5)
  ).take(2)
);
```

А ЕСТЬ ЧТО-ТО КРОМЕ АДАПТЕРОВ?



РАЗУМЕЕТСЯ

- Часто требуется **получить на основе итератора конкретное значение**
- Например, проверить наличие элемента или свернуть последовательность к единому результату
- **Такие задачи решают методы-агрегаторы**
- **В отличие от адаптеров, эти методы «энергичны»:** они запускают итерацию и сразу возвращают конкретное значение, а не новый итератор

```
function random() {
  return Iterator.from({
    next() {
      return { value: Math.random(), done: false };
    }
  });
}

// true
console.log(
  random()
    .filter((value) => value > 0.5)
    .take(10)
    .every((value) => value > 0.5)
);

// 5.575194811834276
console.log(random().take(10).reduce((acc, value) => acc + value));
```

ПОЛНЫЙ СПИСОК

- reduce(fn, initialValue?) – сводит все элементы к одному значению, аналогично Array.prototype.reduce
- some(fn) – проверяет, есть ли хотя бы один элемент, удовлетворяющий условию
- every(fn) – проверяет, все ли элементы удовлетворяют условию
- find(fn) – возвращает первый элемент, для которого fn возвращает true, или undefined, если совпадения нет
- forEach(fn) – выполняет побочный эффект для каждого элемента
- toArray() – собирает все значения итератора в массив


second monitor
$0.00 (0%)
$0.00
224 DAYS TO 20
$160.00
СЛОЖНА, СЛОЖНАААА!

ВЫДЫХАЕМ

- Агрегаторы просто потребляют значения итератора,
  чтобы получить некоторое другое значение
- Главное, что здесь нужно понять, что **агрегаторы «энергичны», а итераторы — нет**

А ЕСЛИ ЗНАЧЕНИЯ ИТЕРАТОРА НУЖНО СОХРАНИТЬ В НЕКОТОРУЮ КОЛЛЕКЦИЮ?



КОЛЛЕКТОРЫ

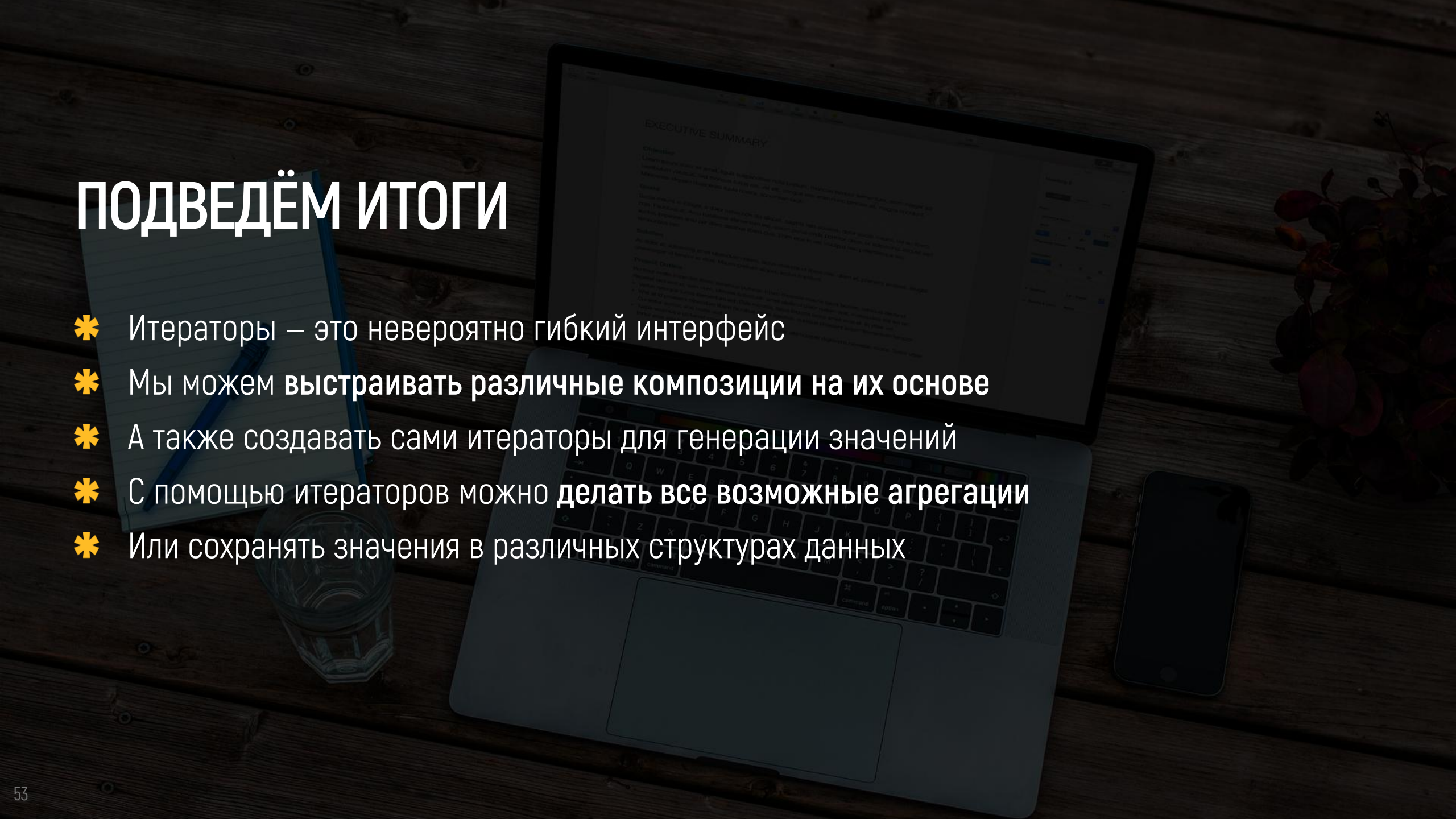
- Функции, которые принимают Iterable
  и **возвращают коллекцию элементов,** называют коллекторами
- Объекты Map и Set умеют создавать себя из Iterable-источника:
  достаточно передать итерируемый объект в конструктор
- Для массивов предусмотрен стандартный способ – функция Array.from
- В работе с промисами роль коллекторов выполняют такие функции, как Promise.all
  и другие: они собирают результаты из набора промисов в единую структуру

РЕФЛЕКСИРУЕМ

- Коллекторы позволяют создавать коллекции **на основе любых Iterable-источников**
- Главное условие – чтобы итератор выдавал данные в нужном формате: тогда один и тот же коллектор можно применять к разным источникам без переписывания логики
- Это существенно **повышает универсальность кода:** абстрагируясь от конкретного типа источника, мы работаем с единым интерфейсом итерации

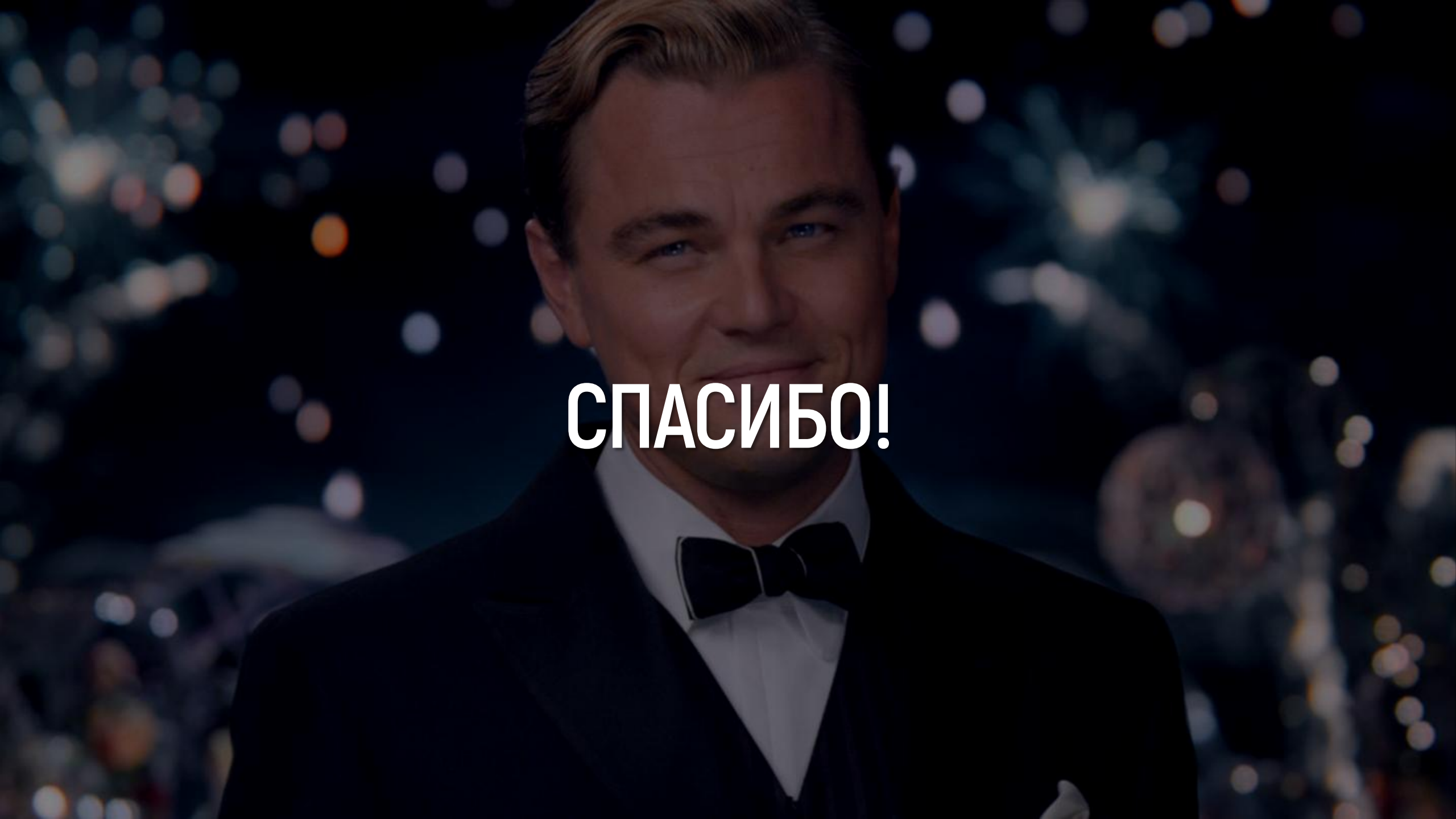
НАМ НЕ ВАЖНА «ПРИРОДА» ИСТОЧНИКА ДАННЫХ

- Можно создать итератор **на основе любых данных**
- Адаптеры **принимают итераторы и возвращают итераторы**
- Агрегаторы принимают итераторы и **возвращают конкретные значения**
- Коллекторы принимают Iterable
  и **возвращают коллекции с элементами итератора**



ПОДВЕДЁМ ИТОГИ

- Итераторы — это невероятно гибкий интерфейс
- Мы можем **выстраивать различные композиции на их основе**
- А также создавать сами итераторы для генерации значений
- С помощью итераторов можно **делать все возможные агрегации**
- Или сохранять значения в различных структурах данных


СПАСИБО!